Design of Compilers CSE247

TINY Language Compiler
Phase (1): Lexical Analysis (Scanner)

Prepared By:

Youssef Atef Ibrahim Elnaggar	23P0341
Mariam Safwat	24P0399
Jana Ahmed Saieed	24P0410
Lujain Mohammed Helmy	24P0285
Youssef Helmy	20P7954

Instructors:

Dr. Wafaa Samy El-Kassas
Dr. Sally Saad Ismail
Eng. Aya
Eng. Youssef Sobhy Ramzy

Table of Contents

What is a Compiler? 4

What is Lexical Analysis (Scanner)? 5

Language Overview 6

- General Structure 6
- Program Components 6
  - Data Types 6
  - Identifiers 6
  - Reserved Keywords 7
  - Constants 7
  - Expressions 7
  - Operators 7
  - Statements 8
  - Control Structure 8
  - Functions 9
  - Main Function 9
  - Comments 9
  - Terms 9
  - Equations 10
- Language Characteristics 10

Token Classification 10

Regular Expressions 11

- Arithmetic Operators 11
- Comment Statements 11
- Condition Operators 11
- Identifiers and Function Name 12
- Numbers 12

Reserved Keywords 12
Strings 13
Data Types 13
Boolean Operators 13
Deterministic Finite Automata (DFA) 14
Arithmetic Operators 14
Comment Statements 14
Condition Operators 15
Identifiers and Function Name 16
Numbers 16
Strings 17
Boolean Operators 17
Reserved Keywords 18
Data Types 19
References 19
Appendix 19

List of Figures

Figure (1): Compilation Process
Figure (2): Lexical Analysis Architecture

List of Tables

Table (1): Tokens and their possible lexemes

What is a Compiler?

A compiler is a software that typically takes up a **high-level language** (Like C++ and Java) code as input and converts the input to a **lower-level language** at once. It lists all the errors if the input code does not follow the rules of its language. This process is much faster than the interpreter, but it becomes difficult to debug all the errors together in a program.

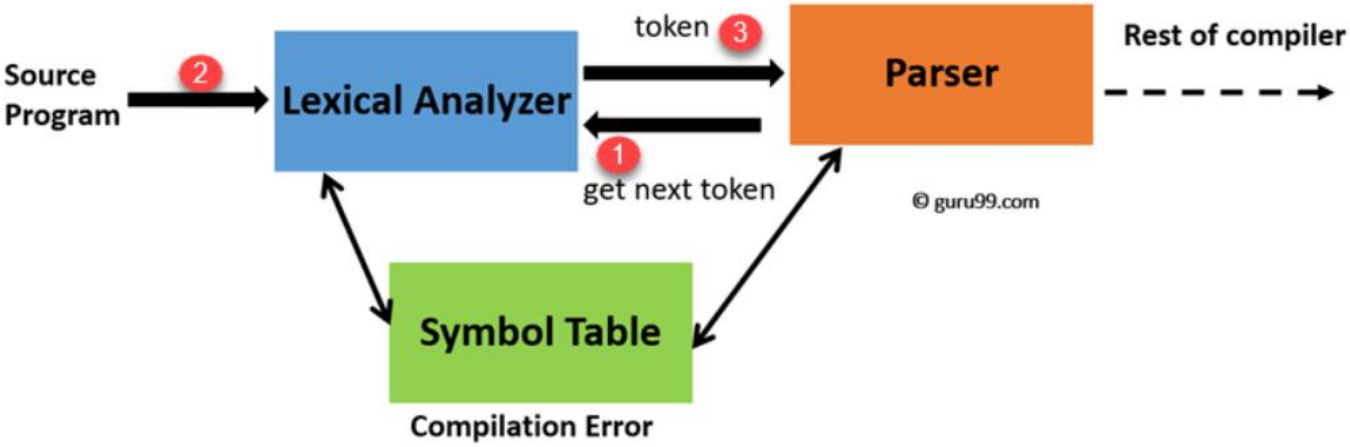
A compiler is a translating program that translates the instructions of high-level language to machine level language. A program which is input to the compiler is called a **Source program**. This program is now converted to a machine level language by a compiler known as the **Object code** [1].

Figure(1): Compilation Process

What is Lexical Analysis (Scanner)?

Lexical Analysis is the very first phase in compiler design. A Lexer takes the modified source code which is written in the form of sentences. In other words, it helps you convert a sequence of characters into a sequence of tokens. The lexical analyzer breaks this syntax into a series of tokens. It removes any extra space or comment written in the source code.

Programs that perform Lexical Analysis in compiler design are called lexical analyzers or lexers. A lexer contains a tokenizer or scanner. If the lexical analyzer detects that the token is invalid, it generates an error. The role of Lexical Analyzer in compiler design is to read character streams from the source code, check for legal tokens, and pass the data to the syntax analyzer when it demands.



Figure(2): Lexical Analysis Architecture

Language Overview

The designed programming language is a simplified high-level language inspired by structured programming principles. It is intended to demonstrate fundamental compiler concepts such as lexical analysis, parsing, and code generation.

General Structure

A program in this language consists of:

- Zero or more **function definitions**
- Followed by a mandatory **main function**

Each function contains a sequence of statements enclosed within curly braces { }.

Program Components

The language supports several types of constructs:

Data Types

The language includes three basic data types:

- int
- float
- String

These are used in **variable declarations** and **function parameters**

Identifiers

Identifiers are used to name variables and functions.

- Must start with a **letter**
- Can contain **letters** and **digits**
- Examples: x, counter1, sum, value2

Reserved Keywords

The language consists of several keywords:

- int
- if
- float
- string
- read
- repeat
- return
- write
- until
- then
- else
- elseif
- end
- endl
- main

Constants

- **Numbers**: Integers and floating-point values
- Examples: 5, 123, 0.75

- **Strings**: Enclosed in double quotes
- Examples: “Hello”, “Test123 + Test456”

Expressions

Expressions can be:

- A **string:** “Hello”
- A **number:** 5
- An **identifier:** x
- An **arithmetic equation:** (2 + 3) * 10

Operators

- Arithmetic Operators: +, -, *, /

- Assignment Operator: :=
- Condition Operators: <, >, =, <>
- Boolean Operators: &&, ||

Statements

The language supports multiple types of statements:

- Declaration Statements: int x;, float a, b := 5;
- Assignment Statements: x := 10;, y := x + 5;
- Read Statements: read x;
- Write Statements: write x;, write "Hello";, write endl;
- Return Statements: return x + 5;

Control Structure

- If Statements
- Example:

```
if x < 5 then
    write x;
end
```

- Elseif / Else Statements
- Example:

```
if x < 5 then
    write x;
elseif x = 5 then
    write 5;
else
    write 0;
end
```

- Repeat Loops
- Example:

```
repeat
    x := x + 1;
until x > 10
```

Functions

Function consists of:

- Declaration
- Body enclosed in {}
- Must end with a return statement
- Example:

```
int sum (int a, int b)
{
    return a + b;
}
```

Main Function

Every program must end with a main function

- Example:

```
int main ()
{
    write "Hello World";
    return 0;
}
```

Comments

- Comments start with /* and end with */
- They are ignored by the scanner
- Example: /* Write a comment here! */

Terms

A term can be:

- **Number:** 56, 6.7
- **Identifier:** x, ABC123
- **Function Call:** sum (int a, int b)

Equations

- Start with Term or (
- Followed by one or more Arithmetic Operator and Term
- End with) for each (
- Examples: 3 + 5, x + 1, (2 +3) * 10

Language Characteristics

- Case-Sensitive (e.g., int ≠ iNT)
- Supports structured programming
- Uses explicit delimiters { }
- Designed for educational compiler construction

Token Classification

Category	Examples
Reserved Keywords	if, int, string, end, endl, else, read,
Data Types	int, float, string
Identifiers	X1, x45, y, counter33
Function Name	Sum, factorial, counter1
Numbers	12, 5, 0.6, 12.5
Strings	"Youssef", "hello", "Me + 45"
Arithmetic Operators	+, -, *, /
Condition Operators	<, >, <>, =
Boolean Operators	&&, ||
Comment	/* This is a comment */

Table(1): Tokens and their possible lexemes

Regular Expressions

Arithmetic Operators

Arithmatic operator RE: (+|-|*|/)

Comment Statements

Comment statement: /*.**/

Condition Operators

- Condition Operator:-

operator:- {<, >, =, <>}

RE: {< | > | = | <>}

Identifiers and Function Name

letter = [a-z A-Z]
digit = [0-9]

Identifier or Function Name = letter (letter | digit)*

Numbers

number = digit + (1·(digit)+)?

→ [0-9]+ (1· [0-9]+)?

Reserved Keywords

Reserved = (int | float | string | read | write | repeat |
until | if | elseif | else | then | return | end | |
end | main |

Strings

STRING = " (letter | digit)* "

" [^ "]* " | " ([a-z] | [A-Z] | [0-9])* "

Data Types

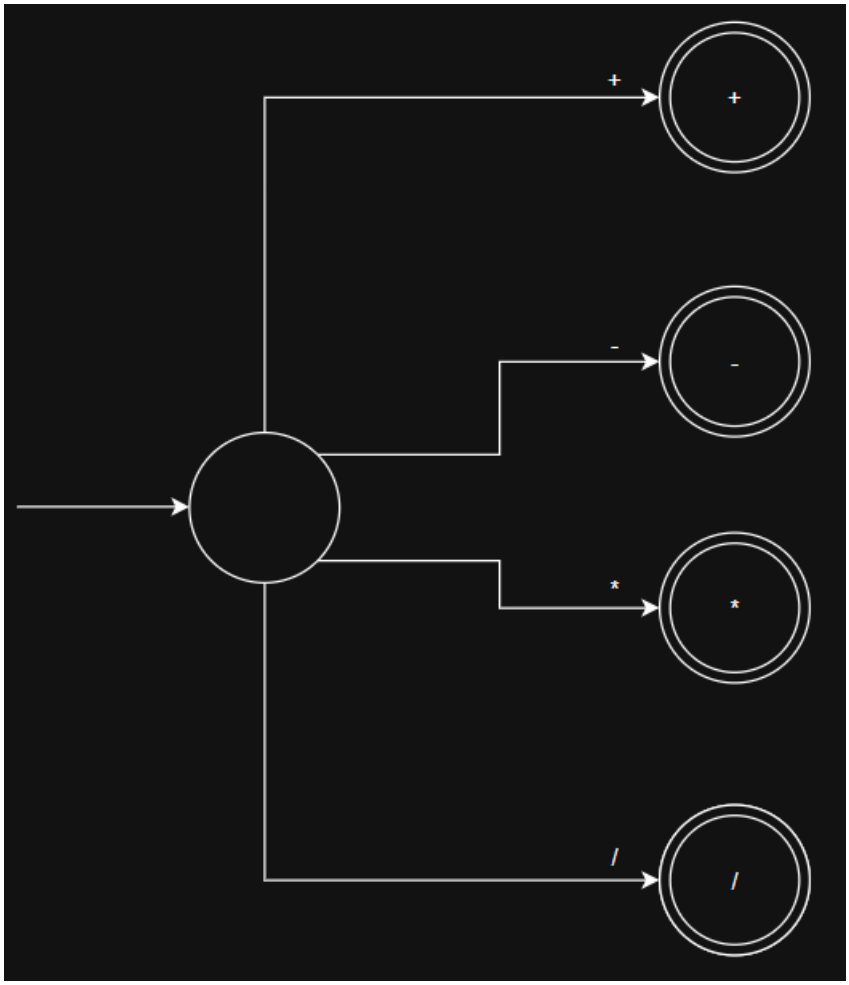
Data Types = (int | float | string)

Boolean Operators

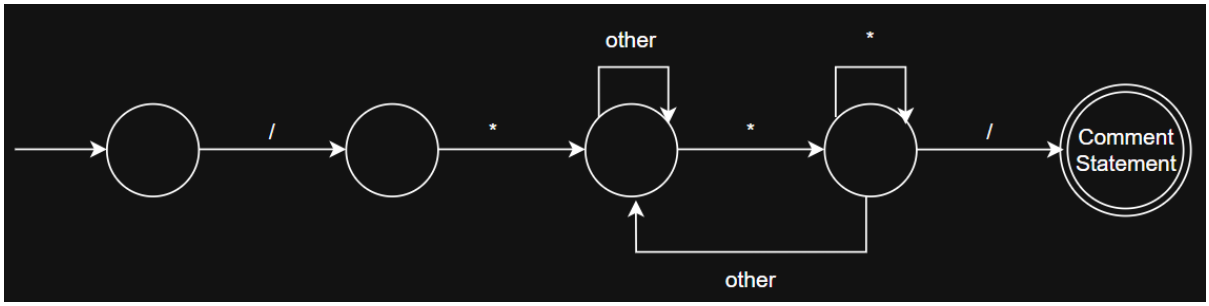
(&& | ||)

Deterministic Finite Automata (DFA)

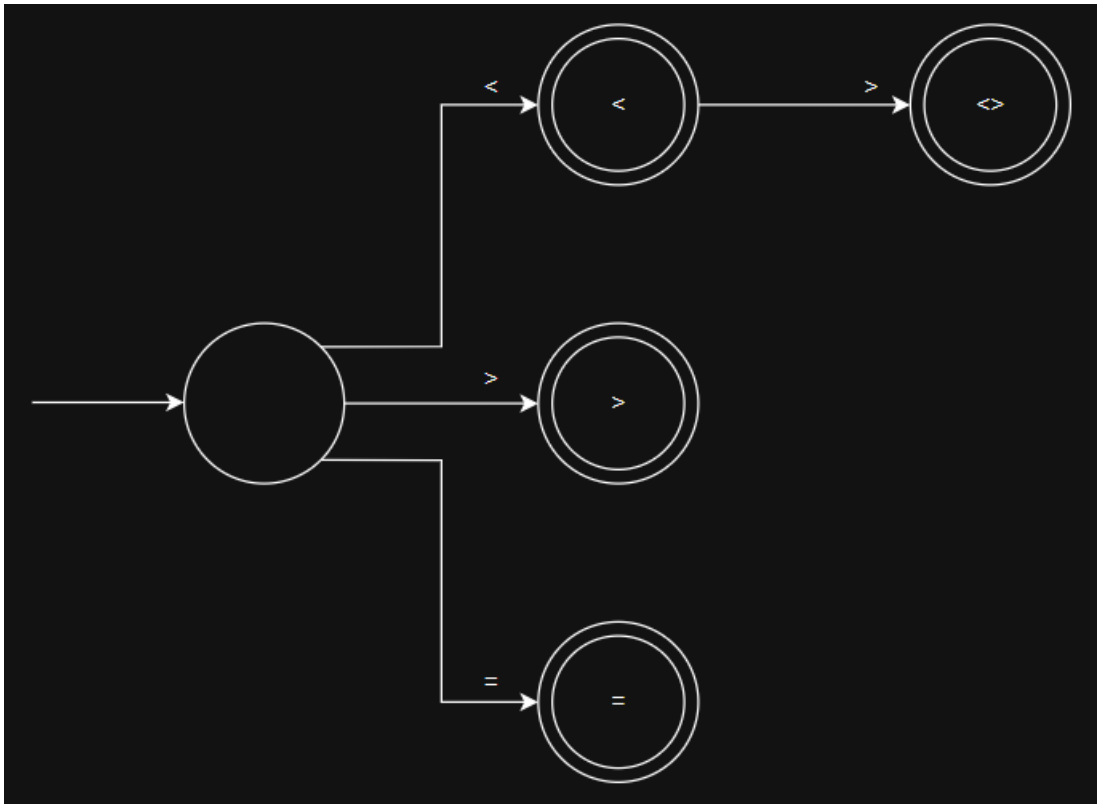
Arithmetic Operators



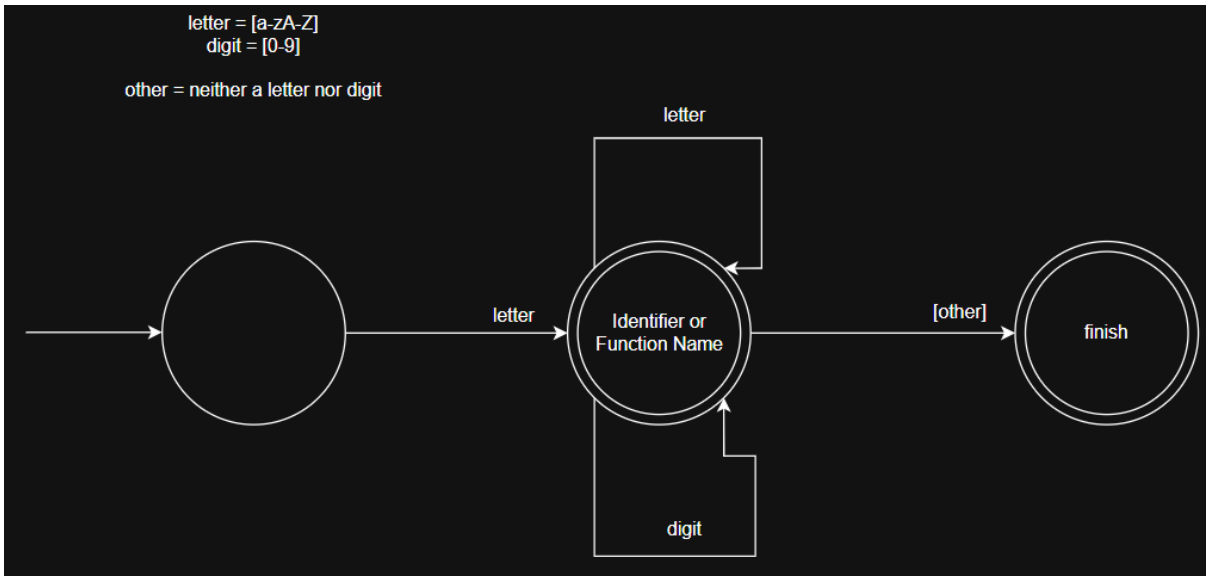
Comment Statements



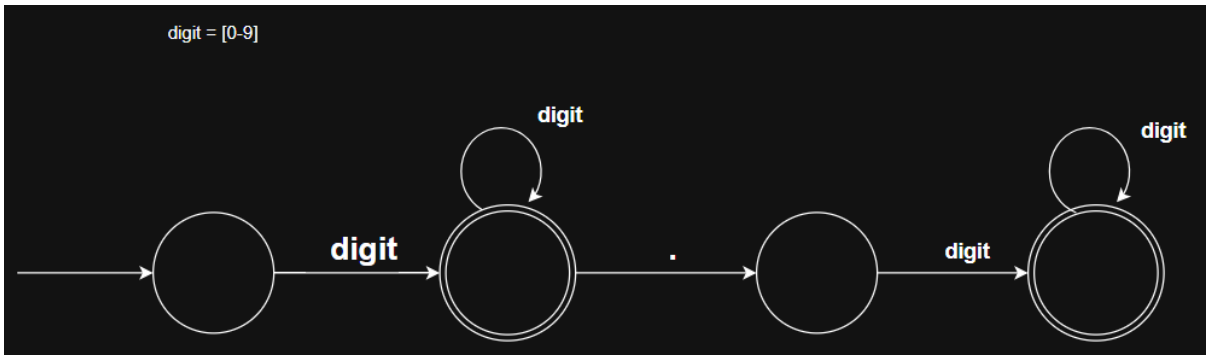
Condition Operators



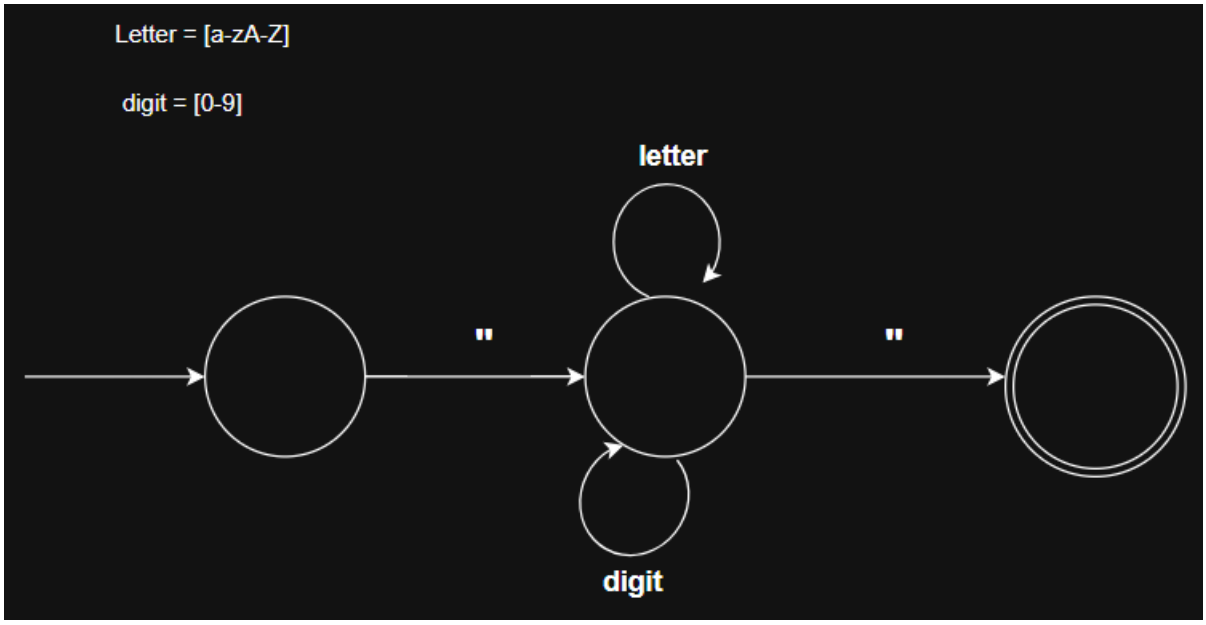
Identifiers and Function Name



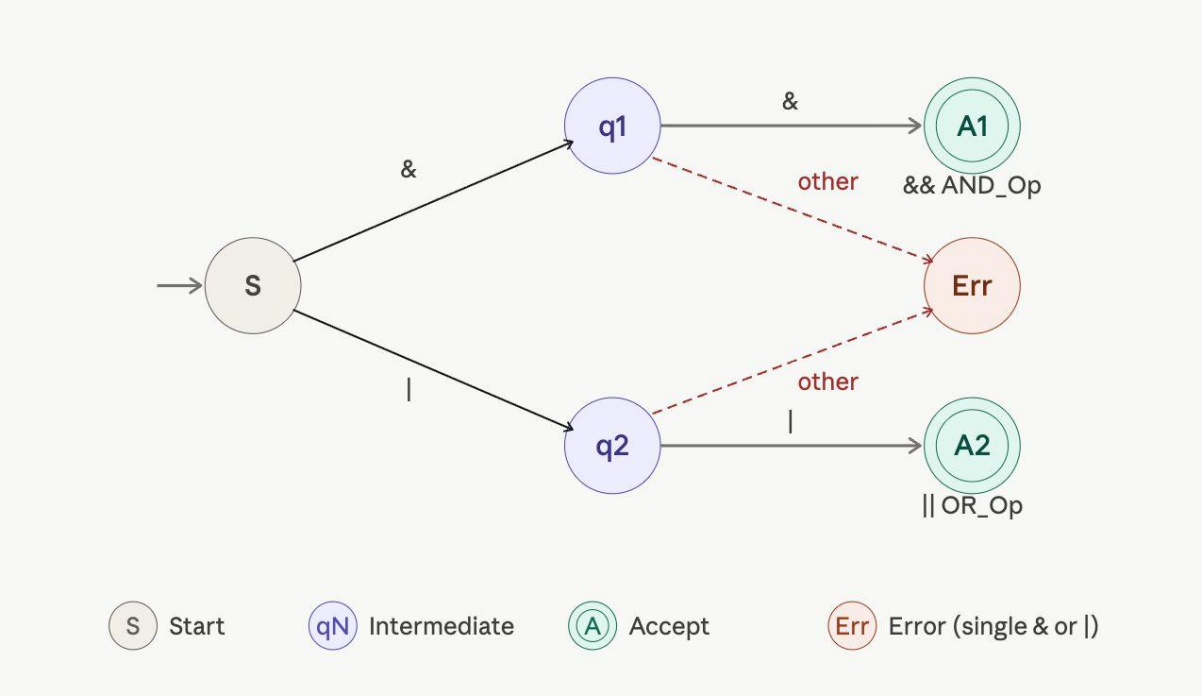
Numbers



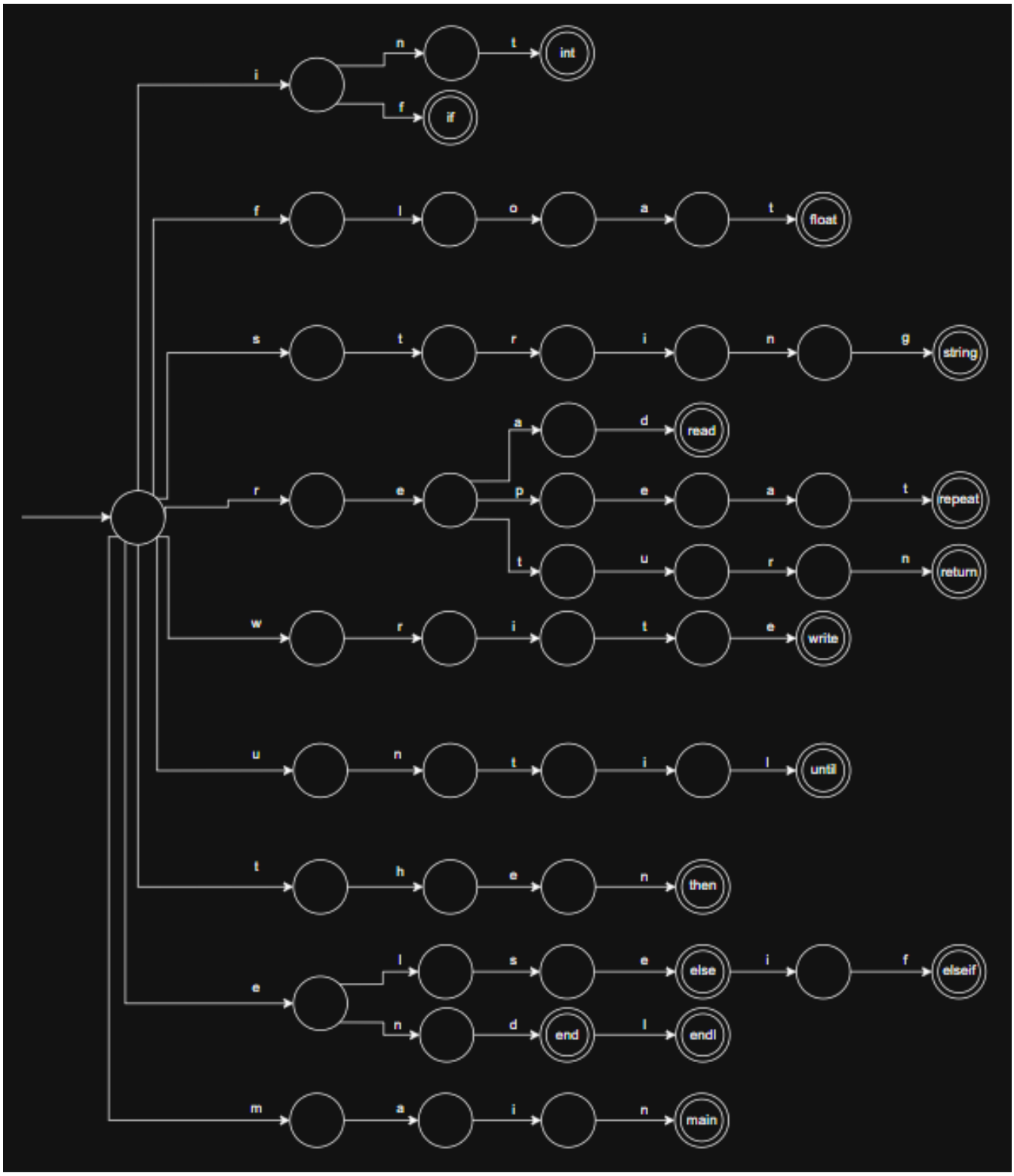
Strings



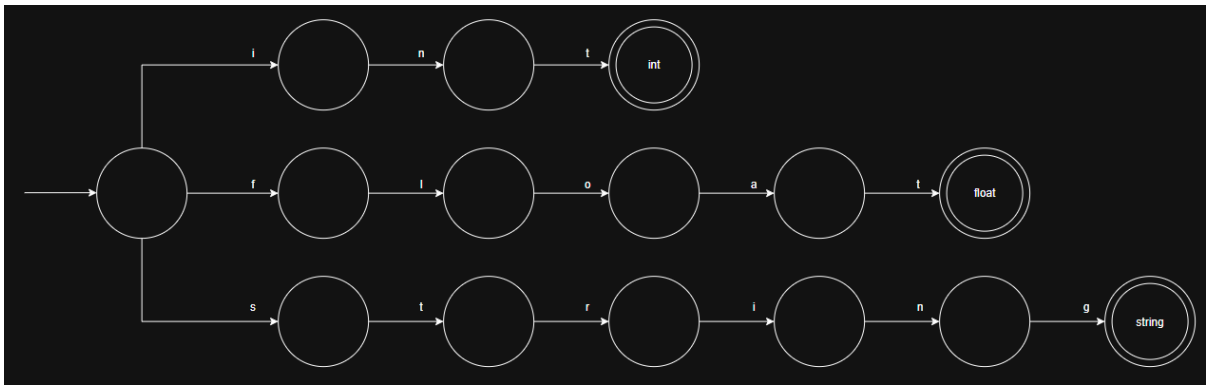
Boolean Operators



Reserved Keywords



Data Types



References

[1] GeeksforGeeks, "Introduction to Compilers," *GeeksforGeeks*. Available: https://www.geeksforgeeks.org/compiler-design/introduction-to-compilers/. Accessed: 13-Apr-2026.

[2] Guru99, "Compiler Design: Lexical Analysis," *Guru99*. Available: https://www.guru99.com/compiler-design-lexical-analysis.html. Accessed: 13-Apr-2026.

Appendix

The complete set of Regular Expressions and DFA diagrams used in the implementation can be found at the following link:

https://drive.google.com/drive/folders/1FETS-1MFciaNdCrRgsW3UrgoD7LVNS4Q?usp=sharing